

Lab Assignment 4: Schema Design

Group Name: The DB Architects

- | | |
|------------------------------|--------------------|
| 1. Name: Mayank Seth | Roll No: 23CS30032 |
| 2. Name: Pratyush Parackal | Roll No: 23CS30040 |
| 3. Name: Siddharth Konnur | Roll No: 23CS30052 |
| 4. Name: Y.V.S.Raghu Vardhan | Roll No: 23CS30060 |

Database Schema Cards

Legend: **Blue** (PK) = Primary Key, **Orange** (FK) = Foreign Key, **Purple** (PK, FK) = Composite/Inherited Key.

Users (Base Table)	
user_id (PK)	Integer
name	Varchar(100)
email	Varchar(100)
password	Varchar(255)
phone_number	Varchar(20)
country	Varchar(50)
gender	Varchar(10)
role_type	Enum
created_at	Timestamp

Universities	
university_id (PK)	Integer
name	Varchar(255)
created_at	Timestamp

Programs	
name (PK)	Varchar(50)
description	TEXT
created_at	Timestamp

Students	
user_id (PK, FK)	Integer
student_roll_id	Varchar(20)
date_of_birth	Date

Instructors	
user_id (PK, FK)	Integer
faculty_id	Varchar(20)
post	Varchar(50)
university_id (FK)	Integer

Courses	
course_id (PK)	Integer
name	Varchar(255)
credits	Integer
description	TEXT
university_id (FK)	Integer
textbook_isbn (FK)	Varchar(20)
created_at	Timestamp

Content	
content_id (PK)	Integer
title	Varchar(255)
url	Text
type	Varchar(20)
created_at	Timestamp

Topics	
topic_id (PK)	Integer
name	Varchar(100)
created_at	Timestamp

Textbooks	
isbn (PK)	Varchar(20)
title	Varchar(255)
author	Varchar(255)
created_at	Timestamp

Taught_By	
instructor_id (PK, FK)	Integer
course_id (PK, FK)	Integer

Program_Courses	
program_name (PK, FK)	Varchar(50)
course_id (PK, FK)	Integer

Has_Content	
course_id (PK, FK)	Integer
content_id (PK, FK)	Integer

Enrolled_In	
student_id (PK, FK)	Integer
course_id (PK, FK)	Integer
marks	Decimal(5,2)
enrollment_date	Timestamp

Course_Topics	
course_id (PK, FK)	Integer
topic_id (PK, FK)	Integer

Prerequisites		
course_id FK)	(PK,	Integer
prereq_id FK)	(PK,	Integer

Student_Skills	
user_id (PK, FK)	Integer
skill_name (PK)	Varchar(50)



Functionalities

1 Introduction

This document outlines the functional specifications of the Online Course Management Platform. The system is a web-based application built using Flask and PostgreSQL, designed to facilitate the interaction between students, instructors, administrators, and data analysts. It manages the entire lifecycle of course registration, content delivery, grading, and academic analysis.

2 System Architecture

The application utilizes a **Role-Based Access Control (RBAC)** system. Upon authentication, users are routed to role-specific dashboards. The backend enforces permission checks using Python decorators (`@login_required` and `@role_required`) to ensure data security.

3 User Modules and Functionalities

3.1 1. Public & Authentication Module

This module handles user entry and security.

- **Landing Page:** Displays live statistics (Total Courses, Students, Instructors) and a showcase of featured courses.
- **User Registration:** Supports creating accounts for Students and Instructors.
 - *Validation:* Checks for duplicate emails.
 - *Role Specifics:* Requires specific metadata during signup (e.g., “Student Roll ID” for students, “Faculty ID” for instructors).
- **Secure Login:** Authenticates users using hashed passwords (via `werkzeug.security`) and establishes a secure session.

3.2 2. Student Module

Students interact with the system to manage their academic progress.

- **Dashboard:**
 - View personal details and current CGPA (calculated dynamically).
 - List currently enrolled courses with real-time marks and grades.
 - View listed skills.

- **Course Catalog & Search:**

- Browse all available courses.
- Search functionality to filter courses by Name or Description.
- Visual indicators showing if the student is already enrolled.

- **Enrollment System:**

- **Prerequisite Verification:** The system strictly enforces prerequisites. A student can only enroll if they have taken the prerequisite course, received a grade, and scored > 50 .

3.3 3. Instructor Module

Instructors manage the courses they are explicitly assigned to.

- **Course Management:** View detailed student lists and upload content (Videos/PDFs) via URL links.
- **Grading System:** Update marks for enrolled students, which immediately reflects in their CGPA.

3.4 4. Administrative Module

The Admin role possesses "Superuser" privileges.

- **System Overview:** View aggregate statistics and recent enrollments.
- **Management:** CRUD operations for Courses and Users, including assigning instructors to courses.
- **Override:** Manually enroll/unenroll students bypassing standard checks.

3.5 5. Data Analyst Module

A read-only role for generating insights.

- **Analytics:** Statistics on enrollment numbers, mark distributions, and instructor workload.
- **Trends:** Visualizing enrollment counts over the last 30 days and program popularity.

Frontend Tools

1 Frontend Architecture Overview

The User Interface (UI) is built on a **Server-Side Rendering (SSR)** architecture using Flask's Jinja2 engine. The visual layer is powered by the **Bootstrap 5 Framework**, utilizing a CDN for performance and standard component classes, with a custom CSS layer for branding.

1.1 Tech Stack

- **Core Framework: Bootstrap 5.3** (via CDN) for grid system, responsive utilities, and pre-built components.
- **Templating Engine:** Jinja2 (Python) for dynamic content injection, layout inheritance via `base.html`, and flash messaging.
- **Styling:** Custom CSS3 overrides (`style.css`) using CSS Variables to theme standard Bootstrap components.
- **Typography:** Inter typeface served via **Google Fonts API** (`fonts.googleapis.com` & `fonts.gstatic.com`).
- **Iconography:** **Bootstrap Icons** (CDN) for vector-based UI elements.
- **Interactivity:** Bootstrap Bundle JS (includes Popper.js) for handling modals, dropdowns, and alerts.

2 Design System Specifications

The design extends the default Bootstrap theme to create a "Modern SaaS" aesthetic.

2.1 Color Palette & Theming

The application overrides Bootstrap defaults using CSS variables defined in `:root`:

- **Primary Brand:** Modern Indigo (`#4F46E5`) replaces the standard Bootstrap Blue.
- **Secondary Brand:** Dark Slate (`#0F172A`) for high-contrast headers.
- **Surface Colors:** Light Gray (`#F3F4F6`) backgrounds to reduce eye strain, contrasting with pure white cards.

2.2 Component Library

The interface leverages standard Bootstrap components with custom styling classes:

2.2.1 1. Navigation

A fixed-top `.navbar` that uses:

- **Responsive Collapse:** Uses `.navbar-toggler` and `.collapse` for mobile menu handling.
- **Session State:** Dynamic Jinja2 logic toggles between "Login/Register" buttons and a "User Dropdown" menu based on `session.user_id`.

2.2.2 2. Alerts & Messaging

Feedback is handled via Flask's `get_flashed_messages()` rendered into Bootstrap Alerts:

- **Dynamic Styling:** The alert category (success, danger) maps directly to Bootstrap classes (e.g., `.alert-danger`).
- **Dismissible:** Includes the generic `.btn-close` handler for user interaction.

2.2.3 3. Card Architecture

Data is presented in `.card` containers that extend Bootstrap:

- **Custom Hover Effects:** Added via `style.css` to provide elevation (shadow depth) on interaction.
- **Structure:** Uses standard `.card-header` and `.card-body` for consistent information hierarchy.

3 Directory Structure

The frontend architecture follows a modular template structure organized by user role.

3.1 Template Organization

All HTML files are located in the `templates/` directory:

- **Core Layout:**
 - `base.html` - The master template containing the `<head>` (CDNs), Navbar logic, and Footer. All other pages extend this file.
 - `index.html` - The public landing page.
- **Authentication:**
 - `login.html` - User login interface.
 - `register.html` - Registration form.
- **Role-Based Dashboards:**
 - `student_*.html` - Student specific views (Courses, Dashboard).
 - `instructor_*.html` - Instructor management tools.
 - `admin_*.html` - Admin CRUD interfaces.
 - `analyst_dashboard.html` - Data reporting view.

4 Responsiveness & UX

- **Grid System:** Utilizes Bootstrap's 12-column grid (`.col-md-8`, `.container`) to ensure layouts adapt from mobile to desktop.
- **Flex Utilities:** Extensive use of `.d-flex`, `.align-items-center`, and `.justify-content-center` for layout alignment.
- **Visual Hierarchy:** Spacing utilities (`.my-5`, `.p-4`) provide breathable whitespace.